

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240090 123240242
NAMA	:	Raihan Buono P Reinnent Rasika Z
PLUG	:	IF-E
NAMA ASISTEN	:	Khatama Putra Bintoro

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Raihan Buono P	123240090
Reinnent Rasika Z	123240242

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Khatama Putra
NIM. 123230053

Asisten Praktikum

Bintoro
NIM. 123230059

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Jogja Tourist Recommendation. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 26 Mei 2026

Penyusun 1,

Penyusun 2,

Raihan Buono P
NIM. 123240090

Reinnent Rasika Z
NIM. 123240242

DAFTAR ISI

HALAMAN PENGESAHAN.....	ii
KATA PENGANTAR.....	iii
DAFTAR ISI.....	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah.....	1
1.2 Tujuan Proyek Akhir.....	1
1.3 Manfaat Proyek Akhir.....	1
BAB II PEMBAHASAN.....	2
2.1 Dasar Teori.....	2
2.2 Skenario Kasus & Dataset.....	2
2.3 Pemodelan Sistem Pendukung Keputusan.....	2
2.4 Perhitungan & Algoritma.....	2
2.5 Implementasi & Tampilan Sistem.....	2
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS.....	3
3.1 Jadwal Pengerjaan.....	3
3.2 Pembagian Tugas.....	3
BAB IV KESIMPULAN DAN SARAN.....	4
4.2 Kesimpulan.....	4
4.3 Saran.....	4
DAFTAR PUSTAKA.....	5

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Yogyakarta memiliki daya tarik pariwisata yang sangat besar, namun banyaknya pilihan destinasi wisata dan akomodasi seringkali membuat wisatawan kesulitan dalam merencanakan perjalanan yang optimal. Wisatawan harus memilah informasi dan mempertimbangkan banyak faktor secara bersamaan, seperti kesesuaian anggaran, kualitas destinasi, serta jarak tempuh antara lokasi menginap dan tempat wisata. Kesulitan ini semakin bertambah ketika wisatawan belum memiliki rencana perjalanan yang terstruktur atau bingung mencocokkan hotel dengan destinasi tujuan mereka.

Sebagai solusi atas permasalahan tersebut, dikembangkan "Jogja Tourism Recommender", yaitu sebuah Sistem Pendukung Keputusan (SPK) untuk rekomendasi destinasi wisata dan akomodasi. Sistem ini menggunakan metode *Weighted Product* (WP) untuk memproses kriteria preferensi pengguna dan mengkalkulasi peringkat alternatif secara matematis. Selain itu, sistem ini mengintegrasikan algoritma *Haversine* guna memberikan analisis spasial yang akurat terkait jarak geospasial antara hotel dan destinasi wisata.

1.2 Tujuan Proyek Akhir

1. Membangun Sistem Pendukung Keputusan (SPK) untuk memberikan rekomendasi destinasi wisata dan akomodasi di Yogyakarta menggunakan metode *Weighted Product* (WP).
2. Menerapkan algoritma *Haversine* untuk perhitungan analisis spasial jarak geospasial secara akurat antara titik hotel dan destinasi wisata.
3. Mengembangkan fitur rekomendasi yang fleksibel untuk memfasilitasi pengguna yang sudah memiliki hotel, belum memiliki hotel, maupun yang membutuhkan paket liburan otomatis.
4. Menciptakan sistem yang dinamis dan responsif sehingga pengguna dapat menyesuaikan bobot kriteria seperti harga tiket, jarak, rating, dan popularitas secara *real-time*.

1.3 Manfaat Proyek Akhir

1. **Efisiensi Perencanaan Perjalanan:** Wisatawan dapat merencanakan perjalanan dengan mudah berdasarkan preferensi pribadi, batasan anggaran (dengan kategori Hemat, Menengah, dan Eksklusif), serta kedekatan lokasi destinasi.

2. **Fleksibilitas Pencarian Akomodasi dan Wisata:** Sistem memberikan kemudahan pencarian dua arah, baik mencari wisata terbaik berdasarkan lokasi menginap saat ini, maupun mencari hotel terbaik di sekitar destinasi wisata yang dituju.
3. **Pengambilan Keputusan yang Objektif dan Stabil:** Penerapan metode WP memberikan hasil perbandingan keputusan yang lebih stabil dan matematis dibandingkan metode aditif sederhana, sehingga rekomendasi yang diberikan lebih akurat.
4. **Analisis Data yang Komprehensif:** Pengguna sangat terbantu dalam membandingkan destinasi melalui dukungan visualisasi data berupa peta interaktif untuk melihat persebaran lokasi dan grafik *Radar Chart* untuk analisis profil atribut wisata atau hotel.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Metode *Weighted Product* (WP) merupakan salah satu metode penyelesaian dalam Sistem Pendukung Keputusan (SPK) yang digunakan untuk mengevaluasi beberapa alternatif terhadap sekumpulan kriteria. Keunggulan utama dari metode ini adalah penggunaan operasi perkalian untuk menghubungkan atribut atau kriteria, di mana setiap nilai kriteria pada suatu alternatif harus dipangkatkan terlebih dahulu dengan bobot kriteria yang bersangkutan. Proses pemangkatan ini menghasilkan keputusan akhir yang lebih stabil dan matematis dibandingkan metode aditif sederhana. Bobot yang digunakan sebagai pangkat bernilai positif untuk kriteria jenis keuntungan (*benefit*) dan bernilai negatif untuk kriteria jenis biaya (*cost*).

2.2 Skenario Kasus & Dataset

Skenario Kasus: Pengambil keputusan dalam sistem ini adalah wisatawan yang sedang merencanakan perjalanan di Yogyakarta. Masalah utama yang dihadapi adalah bagaimana memilih destinasi wisata atau hotel yang paling optimal di tengah banyaknya pilihan, dengan mempertimbangkan batasan anggaran, preferensi kualitas (rating/popularitas), dan efisiensi waktu atau jarak tempuh. Tujuan akhirnya adalah menghasilkan rekomendasi peringkat (ranking) wisata terbaik berdasarkan lokasi hotel tempat wisatawan menginap, atau sebaliknya, rekomendasi hotel terbaik berdasarkan destinasi wisata yang akan dituju.

Dataset: Sistem ini menggunakan dua dataset utama yang telah melalui proses pembersihan (*preprocessing*):

1. `data_hotel_clean.csv`: Berisi informasi mengenai akomodasi yang mencakup atribut Nama Penginapan, Koordinat (Latitude & Longitude), Golongan (Kelas Bintang), dan Jumlah Kamar.
2. `data_wisata_clean.csv`: Berisi informasi mengenai destinasi pariwisata yang mencakup atribut Nama, Koordinat, Rating (*vote_average*), Popularitas (*vote_count*), dan Harga Tiket.

2.3 Pemodelan Sistem Pendukung Keputusan

Berikut adalah pemodelan untuk skenario **Rekomendasi Wisata**.

Daftar Alternatif (Sampel Data Sederhana):

- **A1:** Candi Prambanan
- **A2:** Pantai Parangtritis

- **A3:** Jalan Malioboro

Daftar Kriteria: Dalam menentukan wisata terbaik, digunakan 5 kriteria utama:

- **C1 - Rating (vote_average):** Nilai evaluasi dari pengunjung. Jenis: *Benefit*.
- **C2 - Popularitas (vote_count):** Jumlah ulasan yang diberikan pengunjung. Jenis: *Benefit*.
- **C3 - Harga Tiket:** Biaya masuk destinasi wisata. Jenis: *Cost*.
- **C4 - Jarak dari Hotel:** Jarak geospasial dari lokasi menginap ke wisata. Jenis: *Cost*.
- **C5 - Jarak Pusat Kota:** Jarak destinasi dari pusat kota (Tugu Jogja). Jenis: *Cost*.

Skala Penilaian (Bobot Kriteria): Pengguna (wisatawan) dapat menyesuaikan bobot setiap kriteria menggunakan skala 1 hingga 5:

- 1 = Sangat Rendah / Sangat Tidak Penting
- 2 = Rendah / Tidak Penting
- 3 = Cukup / Sedang
- 4 = Tinggi / Penting
- 5 = Sangat Tinggi / Sangat Penting

2.4 Perhitungan & Algoritma

Langkah-langkah penyelesaian algoritma Weighted Product (WP) yang diimplementasikan dalam sistem adalah sebagai berikut:

1. **Normalisasi Bobot:** Bobot preferensi awal yang diberikan oleh pengguna dinormalisasi sehingga total keseluruhan bobot menjadi 1. Rumusnya adalah dengan membagi bobot suatu kriteria dengan total keseluruhan bobot awal.
2. **Menghitung Vektor S:** Menghitung nilai preferensi untuk setiap alternatif. Nilai setiap kriteria pada suatu alternatif dipangkatkan terlebih dahulu dengan bobot normalisasinya, kemudian hasil pemangkatannya dikalikan. Jika kriteria merupakan atribut benefit, pangkat bernilai positif. Sebaliknya, jika kriteria adalah cost, maka pangkat dikalikan dengan -1 (menjadi negatif). Dalam implementasi sistem, apabila terdapat nilai 0 pada data (seperti harga tiket gratis atau jarak 0 km), nilai tersebut ditransformasi menjadi nilai toleransi 0.01 untuk mencegah error matematis (nilai menjadi 0) pada operasi perkalian.
3. **Menghitung Vektor V:** Menghitung nilai preferensi relatif dari setiap alternatif. Nilai Vektor S dari sebuah alternatif spesifik dibagi dengan total akumulasi nilai Vektor S dari keseluruhan alternatif.
4. **Perangkingan:** Nilai Vektor V yang dihasilkan kemudian diurutkan (sorting) dari nilai terbesar ke terkecil (descending). Alternatif dengan nilai Vektor V tertinggi menempati peringkat pertama dan direkomendasikan sebagai pilihan terbaik.

2.5 Implementasi & Tampilan Sistem

```
# main.py
from pathlib import Path
import pandas as pd
import numpy as np
# Mengambil rumus dari file wp.py
from wp import core_weighted_product, hitung_jarak_haversine

# Setup Path Data Sesuai Kepunyaanmu
CURRENT_DIR = Path(__file__).resolve().parent
PROJECT_ROOT = CURRENT_DIR.parent

CSV_PATH_HOTEL = PROJECT_ROOT / "dataset" / "clean" / "data_hotel_clean.csv"
CSV_PATH_PARIWISATA = PROJECT_ROOT / "dataset" / "clean" / "data_pariwisata_clean.csv"
CSV_PATH_WISATA_FINAL = PROJECT_ROOT / "dataset" / "clean" / "data_wisata_clean.csv"

# Load global data
df_hotel = pd.read_csv(CSV_PATH_HOTEL)
df_pariwisata = pd.read_csv(CSV_PATH_PARIWISATA)
df_wisata_final = pd.read_csv(CSV_PATH_WISATA_FINAL)

# Merge jarak_pusat_km + jumlah_hotel_terdekat dari data_pariwisata ke data_wisata_final
df_wisata_final = df_wisata_final.merge(
    df_pariwisata[['nama', 'jarak_pusat_km', 'jumlah_hotel_terdekat']],
    on='nama', how='left'
)
df_wisata_final['jarak_pusat_km'] =
df_wisata_final['jarak_pusat_km'].fillna(df_wisata_final['jarak_pusat_km'].median())
df_wisata_final['jumlah_hotel_terdekat'] =
df_wisata_final['jumlah_hotel_terdekat'].fillna(0)

# FUNGSI REKOMENDASI UTAMA & REKOMENDASI DINAMIS

def rekomendasi_wisata_dari_hotel(nama_hotel, bobot_user,
    budget_maks=None, kategori=None, keyword=None, sort_order='descending',
    hari='Weekday'):
    """Mode A: Rekomendasi Wisata + Multi-filtering + Harga Dinamis"""
    hotel_terpilih = df_hotel[df_hotel['NAMA PENGINAPAN'] ==
nama_hotel].iloc[0]

    df_dinamis = df_wisata_final.copy()

    # Hitung Jarak ke Hotel secara real-time
    df_dinamis['jarak_ke_hotel'] = hitung_jarak_haversine(
        hotel_terpilih['Latitude'], hotel_terpilih['Longitude'],
        df_dinamis['latitude'], df_dinamis['longitude']
    )

    # Kolom harga dinamis berdasarkan pilihan hari
    harga_col = 'htm_weekday' if hari == 'Weekday' else 'htm_weekend'
    df_dinamis['harga tiket'] = df_dinamis[harga_col]
```

```

# Filter budget menggunakan harga dinamis
if budget_maks:
    df_dinamis = df_dinamis[df_dinamis['harga_tiket'] <= budget_maks]
if kategori and kategori.lower() != 'semua':
    df_dinamis = df_dinamis[df_dinamis['type'].str.lower() ==
kategori.lower()]
if keyword:
    df_dinamis = df_dinamis[
        df_dinamis['nama'].str.contains(keyword, case=False) |
        df_dinamis['description'].str.contains(keyword, case=False)
    ]

# 5 Kriteria: rating, popularitas, harga (dinamis), jarak hotel,
jarak pusat kota
jenis_kri = {
    'vote_average': 'benefit',
    'vote_count': 'benefit',
    'harga_tiket': 'cost',
    'jarak_ke_hotel': 'cost',
    'jarak_pusat_km': 'cost',
}

return core_weighted_product(df_dinamis, bobot_user, jenis_kri,
sort_order)

def rekomendasi_hotel_dari_wisata(nama_wisata, bobot_user,
sort_order='descending'):
    """Mode B: Rekomendasi Hotel terbaik berdasarkan objek wisata
    terpilih (4 Kriteria)"""
    wisata_terpilih = df_wisata_final[df_wisata_final['nama'] ==
nama_wisata].iloc[0]

    df_dinamis = df_hotel.copy()

    # Konversi teks kelas hotel menjadi angka (1-5)
    kelas_rank = {
        'Bintang Lima': 5, 'Bintang Empat': 4, 'Bintang Tiga': 3,
        'Bintang Dua': 2, 'Bintang Satu': 1, 'Melati': 1,
        'Vila': 1, 'Pondok Wisata': 1, 'Akomodasi Lain': 1
    }

    df_dinamis['GOLONGAN_SCORE'] =
df_dinamis['GOLONGAN'].map(kelas_rank).fillna(1)
    df_dinamis['JUMLAH KAMAR'] = df_dinamis['JUMLAH
KAMAR'].fillna(df_dinamis['JUMLAH KAMAR'].median())

    df_dinamis['jarak_ke_wisata'] = hitung_jarak_haversine(
        wisata_terpilih['latitude'], wisata_terpilih['longitude'],
        df_dinamis['Latitude'], df_dinamis['Longitude']
    )
    df_dinamis['estimasi_waktu menit'] = (df_dinamis['jarak_ke_wisata'] /
40) * 60

# Menggunakan 4 kriteria hotel yang efisien
jenis_kri = {
    'JUMLAH KAMAR': 'benefit',
    'GOLONGAN_SCORE': 'benefit',
    'jarak_ke_wisata': 'cost',
    'estimasi_waktu menit': 'cost',

```

```

    }

    # Mencegah error jika UI mengirimkan bobot berlebih
    bobot_aktif = {k: v for k, v in bobot_user.items() if k in jenis_kri}

    return core_weighted_product(df_dinamis, bobot_aktif, jenis_kri,
sort_order)

def rekomendasi_wisata_global(bobot_user, kategori=None, keyword=None,
budget_min=0, budget_maks=None, sort_order='descending'):
    """Mode C: Rekomendasi Wisata dengan rentang harga batas bawah dan
atas"""
    df_dinamis = df_wisata_final.copy()

    # Pakai harga standar weekday
    df_dinamis['harga_tiket'] = df_dinamis['htm_weekday']

    # UPDATE LOGIC: Filter Rentang Budget (Min & Max)
    if budget_maks is not None:
        df_dinamis = df_dinamis[
            (df_dinamis['harga_tiket'] > budget_min) &
            (df_dinamis['harga_tiket'] <= budget_maks)
        ]

    if kategori and kategori.lower() != 'semua':
        df_dinamis = df_dinamis[df_dinamis['type'].str.lower() ==
kategori.lower()]
    if keyword:
        df_dinamis = df_dinamis[
            df_dinamis['nama'].str.contains(keyword, case=False) |
            df_dinamis['description'].str.contains(keyword, case=False)
        ]

    jenis_kri = {
        'harga_tiket': 'cost',
        'jarak_pusat_km': 'cost',
        'jumlah_hotel_terdekat': 'benefit',
        'vote_average': 'benefit',
        'vote_count': 'benefit',
    }

    bobot_aktif = {k: v for k, v in bobot_user.items() if k in jenis_kri}

    return core_weighted_product(df_dinamis, bobot_aktif, jenis_kri,
sort_order)

def buat_paket_itinerary(nama_hotel, bobot_user, radius_km=15):
    """Membuat paket rute 3 wisata terdekat dari hotel"""
    hotel_terpilih = df_hotel[df_hotel['NAMA PENGINAPAN'] ==
nama_hotel].iloc[0]
    df_dinamis = df_pariwisata.copy()
    df_dinamis['jarak_ke_hotel'] = hitung_jarak_haversine(
        hotel_terpilih['Latitude'], hotel_terpilih['Longitude'],
df_dinamis['latitude'], df_dinamis['longitude']
    )

    df_radius = df_dinamis[df_dinamis['jarak_ke_hotel'] <= radius_km]
    jenis_kri = {

```

```

        'vote_average': 'benefit', 'vote_count': 'benefit',
        'htm_weekday': 'cost', 'htm_weekend': 'cost', 'jarak_ke_hotel':
'cost'
    }
    hasil = core_weighted_product(df_radius, bobot_user, jenis_kri,
sort_order='descending')
    return hasil.head(3)

def hitung_analisis_sensitivitas(nama_hotel, bobot_base, kriteria_diubah,
delta=1, hari='Weekday'):
    """Melihat efek pergeseran ranking jika bobot kriteria diubah"""
    df_awal = rekomendasi_wisata_dari_hotel(nama_hotel, bobot_base,
hari=hari)
    df_awal_sub = df_awal[['nama', 'Ranking']].rename(columns={'Ranking':
'Rank_Awal'})

    bobot_baru = bobot_base.copy()
    bobot_baru[kriteria_diubah] += delta

    df_baru = rekomendasi_wisata_dari_hotel(nama_hotel, bobot_baru,
hari=hari)
    df_baru_sub = df_baru[['nama', 'Ranking']].rename(columns={'Ranking':
'Rank_Baru'})

    merge_df = pd.merge(df_awal_sub, df_baru_sub, on='nama')
    merge_df['Perubahan_Posisi'] = merge_df['Rank_Awal'] -
merge_df['Rank_Baru']
    return merge_df.sort_values(by='Rank_Baru').head(10)

# ui_components.py
import streamlit as st
import plotly.graph_objects as go
from streamlit_extras.metric_cards import style_metric_cards

def min_max_scale(val, min_val, max_val, is_cost=False):
    """Fungsi pembantu untuk normalisasi data radar chart"""
    if max_val == min_val: return 100
    if is_cost: return ((max_val - val) / (max_val - min_val)) * 100
    return ((val - min_val) / (max_val - min_val)) * 100

def ui_header():
    col1, col2 = st.columns([3, 1])
    with col1:
        st.title("🏞️ Jogja Tourism Recommender")
        st.caption("Sistem Pendukung Keputusan · Metode Weighted Product
· Reinnett Rasika Z")
    with col2:
        st.metric("Dataset Hotel", "X hotel")
        st.metric("Destinasi Wisata", "X tempat")

def ui_section(text, sub=""):
    """Judul section bawaan Streamlit"""
    st.subheader(text)
    if sub:
        st.caption(sub)

def ui_card_juara(peringkat, kategori, nama, skor, is_hotel=False):
    """Card Juara menggunakan Container dan alert bawaan Streamlit"""
    with st.container(border=True):
        if is_hotel:

```

```

        st.warning(f"🏆 **Peringkat {peringkat} | {kategori}**")
    else:
        st.info(f"🏆 **Peringkat {peringkat} | {kategori}**")

    st.header(nama)
    st.write(f"**Skor WP:** {skor:.6f}")

def ui_card_paket(index, w_row, hotel_cocok):
    """Card Paket Liburan menggunakan Column dan Container bawaan"""
    bintang_paket = "★" * int(hotel_cocok['GOLONGAN_SCORE'])

    with st.container(border=True):
        st.subheader(f"🎯 Opsi Paket #{index}")

        col1, col2 = st.columns(2)
        with col1:
            st.success(f"**Destinasi Utama:**\n### {w_row['nama']}")
            st.write(f"★ Rating: {w_row['vote_average']} | 💰 Tiket: Rp {int(w_row['harga_tiket']):,}")

        with col2:
            st.warning(f"**Penginapan Terdekat:**\n### {hotel_cocok['NAMA PENGINAPAN']}")
            st.write(f"📍 Jarak: {hotel_cocok['jarak_ke_wisata']:.2f} km | 🏠 Kelas: {bintang_paket}")

def draw_radar_chart(df_hasil, labels, keys, costs):
    """Membangun Grafik Radar menggunakan Plotly (Python)"""
    max_df, min_df = df_hasil.max(), df_hasil.min()
    fig = go.Figure()
    colors = [("#6359FF", "rgba(99,89,255,0.15)"), ("00D2A0", "rgba(0,210,160,0.12)")]

    for i, (cl, cf) in enumerate(colors):
        if i >= len(df_hasil): break
        row = df_hasil.iloc[i]
        skala = [min_max_scale(row[k], min_df[k], max_df[k], is_cost=costs[idx]) for idx, k in enumerate(keys)]

        # PERBAIKAN: Ambil nama berdasarkan jenis data (Wisata atau Hotel) jadi string
        item_name = row['nama'] if 'nama' in row else row['NAMA PENGINAPAN']

        fig.add_trace(go.Scatterpolar(
            r=skala + [skala[0]], theta=labels + [labels[0]],
            fill='toself', fillcolor=cf, line=dict(color=cl, width=2),
            name=f"#{i+1} {str(item_name)[:22]}"
        ))

    fig.update_layout(
        polar=dict(
            radialaxis=dict(visible=True, range=[0,100]),
        ),
        showlegend=True,
        margin=dict(l=40, r=40, t=20, b=40), height=350
    )
    st.plotly_chart(fig, use_container_width=True)

```

```

# app.py – SPK Wisata DIY · Native Streamlit
import streamlit as st
import pandas as pd
import folium
from streamlit_folium import st_folium
import matplotlib.pyplot as plt
import seaborn as sns

try:
    from main import (
        df_hotel, df_wisata_final, df_pariwisata,
        rekomendasi_wisata_dari_hotel,
        rekomendasi_hotel_dari_wisata,
        rekomendasi_wisata_global,
        hitung_analisis_sensitivitas,
    )
    from ui_components import draw_radar_chart
except ImportError as e:
    st.error(f"Gagal memuat modul: {e}")
    st.stop()

st.set_page_config(
    page_title="Jogja Tourism Recommender",
    page_icon="🏰",
    layout="wide",
    initial_sidebar_state="expanded",
)

# SESSION STATE
if "active_mode" not in st.session_state: st.session_state.active_mode = "Cari Pariwisata"
if "target_wisata" not in st.session_state: st.session_state.target_wisata = None
if "hasil_paket" not in st.session_state: st.session_state.hasil_paket = pd.DataFrame()

# HELPER: SEARCH SELECTBOX
def search_selectbox(label, options, default_value=None, key_prefix=""):
    """Text input + filtered selectbox – meniru perilaku autocomplete."""
    query = st.text_input(label, placeholder="Ketik nama untuk mencari...", key=f"{key_prefix}_query")
    filtered = [o for o in options if query.lower() in o.lower()] if query else options

    if not filtered:
        st.caption("Tidak ada hasil yang cocok.")
        return default_value

    # Tentukan index default
    def_idx = 0
    if default_value and default_value in filtered:
        def_idx = filtered.index(default_value)

    return st.selectbox(
        f"Hasil pencarian ({len(filtered)} ditemukan)" if query else "Pilih dari daftar",
        filtered,
        index=def_idx,
    )

```

```

        key=f"{key_prefix}_select",
        label_visibility="collapsed",
    )

# POPUP DIALOG
@st.dialog("Detail Destinasi")
def popup_detail_wisata(w_row):
    st.subheader(w_row["nama"])
    a, b = st.columns(2)
    a.metric("Kategori", w_row.get("type", "-"))
    b.metric("Rating", f"{w_row['vote_average']:.1f} / 5")
    a.metric("Harga Tiket", f"Rp {int(w_row['harga_tiket']):,}")
    b.metric("Jarak Pusat Kota", f"{w_row['jarak_pusat_km']:.1f} km")
    a.metric("Ulasan", f"{int(w_row['vote_count']):,} orang")
    b.metric("Hotel Terdekat",
f"{int(w_row.get('jumlah_hotel_terdekat', 0))} hotel")
    st.divider()
    st.info("Ingin melihat rekomendasi hotel khusus untuk destinasi
ini?")
    if st.button("Cari Hotel untuk Destinasi Ini", type="primary",
use_container_width=True):
        st.session_state.active_mode = "Cari Hotel"
        st.session_state.target_wisata = w_row["nama"]
        st.rerun()

#SIDEBAR
with st.sidebar:
    st.header("Panel Kendali")

    mode_pilih = st.radio(
        "mode",
        ["Cari Pariwisata", "Cari Hotel", "Belum Ada Tujuan", "Data &
Analitik"], # <-- UBAH DI SINI
        key="active_mode",
        label_visibility="collapsed",
    )

    MODE_A = mode_pilih == "Cari Pariwisata"
    MODE_B = mode_pilih == "Cari Hotel"
    MODE_C = mode_pilih == "Belum Ada Tujuan"
    MODE_D = mode_pilih == "Data & Analitik"
    st.divider()

    # Default semua variabel
    hotel_pilih = wisata_pilih = None
    hari_pilih = "Weekday"
    radius_input = 15.0
    budget_min, budget_maks = 0, 100000
    kat_pilih = "Semua"
    keyword_pilih = ""
    bobot_user = bobot_hotel = bobot_wisata_global = {}
    bobot_hotel_global = {"JUMLAH KAMAR": 3, "GOLONGAN_SCORE": 3,
"jarak_ke_wisata": 5, "estimasi_waktu_menit": 4}
    filter_bintang_b = filter_bintang_c = [1, 2, 3, 4, 5]

    #MODE A
    if MODE_A:

```

```

        st.caption("HOTEL MENGINAP")
        hotel_list = sorted(df_hotel["NAMA
PENGINAPAN"].dropna().unique().tolist())
        hotel_pilih = search_selectbox("Cari hotel", hotel_list,
key_prefix="hotel_a")

        st.divider()
        radius_input = st.slider("Radius maksimal (km)", 1.0, 50.0, 15.0,
0.5)
        hari_pilih = st.radio("Hari kunjungan", ["Weekday", "Weekend"],
horizontal=True)

        st.divider()
        st.caption("FILTER WISATA")
        kat_pilih = st.selectbox("Kategori", ["Semua"] +
df_wisata_final["type"].dropna().unique().tolist())
        budget_maks = st.number_input("Budget tiket maks (Rp)",
min_value=0, value=100000, step=5000)
        keyword_pilih= st.text_input("Kata kunci", placeholder="Contoh:
Candi, Pantai")

        st.divider()
        with st.expander("Bobot Kriteria (1-5)"):
            st.caption("Geser untuk menyesuaikan prioritas.")
            bobot_user = {
                "vote_average": st.slider("Rating", 1, 5,
4),
                "vote_count": st.slider("Popularitas", 1, 5,
3),
                "harga_tiket": st.slider("Harga Tiket", 1, 5,
4),
                "jarak_ke_hotel": st.slider("Jarak dari Hotel", 1, 5,
5),
                "jarak_pusat_km": st.slider("Jarak Pusat Kota", 1, 5,
2),
            }

        # MODE B
        elif MODE_B:
            st.caption("TUJUAN WISATA")
            wisata_list =
sorted(df_wisata_final["nama"].dropna().unique().tolist())
            wisata_pilih = search_selectbox(
                "Cari wisata", wisata_list,
                default_value=st.session_state.target_wisata,
                key_prefix="wisata_b",
            )

            st.divider()
            st.caption("FILTER KELAS HOTEL")
            filter_bintang_b = st.multiselect(
                "Kelas hotel",
                options=[1, 2, 3, 4, 5],
                default=[1, 2, 3, 4, 5],
                format_func=lambda x: f"Kelas {x}",
            )

            st.divider()
            with st.expander("Bobot Kriteria Hotel (1-5)"):

```



```

        bobot_hotel = {
            "JUMLAH KAMAR":          st.slider("Kapasitas Kamar", 1,
5, 3),
            "GOLONGAN_SCORE":        st.slider("Kelas Hotel", 1,
5, 4),
            "jarak_ke_wisata":        st.slider("Jarak ke Wisata", 1,
5, 5),
            "estimasi_waktu_menit": st.slider("Estimasi Waktu", 1,
5, 4),
        }

#bODE C
elif MODE_C:
    st.caption("GAYA LIBURAN")
    gaya = st.selectbox(
        "Budget wisata",
        ["Hemat (< Rp 10.000)", "Menengah (Rp 10-50 ribu)",
"Eksklusif (> Rp 50.000)"],
    )
    if gaya.startswith("Hemat"):
        budget_min, budget_maks, def_w, bh_bintang = -1, 10000,
[5,4,3,4,3], 1
    elif gaya.startswith("Menengah"):
        budget_min, budget_maks, def_w, bh_bintang = 10000, 50000,
[3,3,4,4,4], 3
    else:
        budget_min, budget_maks, def_w, bh_bintang = 50000,
1_000_000, [1,2,5,5,5], 5

    st.divider()
    st.caption("FILTER DESTINASI")
    kat_pilih = st.selectbox("Kategori", ["Semua"] +
df_wisata_final["type"].dropna().unique().tolist())
    keyword_pilih = st.text_input("Kata kunci", placeholder="Contoh:
Candi, Pantai")
    filter_bintang_c = st.multiselect(
        "Kelas hotel",
        options=[1, 2, 3, 4, 5],
        default=[1, 2] if gaya.startswith("Hemat") else ([3] if
gaya.startswith("Menengah") else [4, 5]),
        format_func=lambda x: f"Kelas {x}",
    )

    st.divider()
    with st.expander("Bobot Kriteria Wisata (1-5)":
        bobot_wisata_global = {
            "harga_tiket":          st.slider("Harga Tiket",
1, 5, def_w[0]),
            "jarak_pusat_km":        st.slider("Jarak Pusat Kota",
1, 5, def_w[1]),
            "jumlah_hotel_terdekat": st.slider("Hotel Terdekat",
1, 5, def_w[2]),
            "vote_average":          st.slider("Rating",
1, 5, def_w[3]),
            "vote_count":            st.slider("Popularitas",
1, 5, def_w[4]),
        }
        bobot_hotel_global = {
            "JUMLAH KAMAR": 3, "GOLONGAN_SCORE": bh_bintang,

```

```

        "jarak_ke_wisata": 5, "estimasi_waktu_menit": 4,
    }
    elif MODE_D:
        st.info("👉 Silakan lihat halaman utama (sebelah kanan) untuk
melihat data dan analitik.")

# HEADER
col_ttl, col_stat = st.columns([3, 1])
with col_ttl:
    st.title("Jogja Tourism – Recommender")
    st.caption("Sistem Pendukung Keputusan · Metode Weighted Product ·
Reinment Rasika Z & Tim")
with col_stat:
    with st.container(border=True):
        a, b = st.columns(2)
        a.metric("Hotel", f"{len(df_hotel):,}")
        b.metric("Wisata", f"{len(df_wisata_final):,}")

st.divider()

# — TABS

if MODE_A:
    tab1, tab2, tab3 = st.tabs(["Rekomendasi Wisata", "Peta Lokasi",
"Sensitivitas Bobot"])
elif MODE_B:
    tab1, tab2 = st.tabs(["Rekomendasi Hotel", "Peta Lokasi"])
elif MODE_C:
    tab1, tab2 = st.tabs(["Paket Liburan", "Peta Lokasi"])
elif MODE_D:
    tab_profil, tab_data, tab_grafik = st.tabs(["👥 Profil Kelompok",
"📁 Dataset Mentah", "📊 Visualisasi Analitik"])

# MODE A

if MODE_A:
    with tab1:
        st.subheader("Rekomendasi Wisata Terbaik")
        if hotel_pilih:
            st.caption(f"Hotel: **{hotel_pilih}** · Radius:
**{radius_input} km** · Hari: **{hari_pilih}**")
        else:
            st.warning("Pilih hotel terlebih dahulu di panel kiri.")

            if hotel_pilih and st.button("Hitung Rekomendasi",
type="primary", use_container_width=True):
                with st.spinner("Memproses algoritma Weighted Product..."):
                    hasil_mentah = rekomendasi_wisata_dari_hotel(
                        hotel_pilih, bobot_user, budget_maks, kat_pilih,
keyword_pilih, "descending", hari_pilih
                    )
                    hasil = hasil_mentah[hasil_mentah["jarak_ke_hotel"] <=
radius_input]

                    if hasil.empty:

```

```

        st.warning(f"Tidak ada destinasi dalam radius
{radius_input} km dengan budget yang dipilih.")
    else:
        juara = hasil.iloc[0]
        with st.container(border=True):
            kol_badge, _ = st.columns([1, 4])
            kol_badge.success("Peringkat #1")
            st.subheader(juara["nama"])
            st.caption(f"{juara.get('type','-')} · Skor WP:
{juara['Vector_V']:.6f}")
            m1, m2, m3, m4, m5 = st.columns(5)
            m1.metric("Rating",
f"{juara['vote_average']:.1f} / 5")
            m2.metric("Ulasan",
f"{int(juara['vote_count']):,}")
            m3.metric("Harga",
f"Rp
{int(juara['harga_tiket']):,}")
            m4.metric("Jarak Hotel",
f"{juara['jarak_ke_hotel']:.1f} km")
            m5.metric("Jarak Pusat",
f"{juara['jarak_pusat_km']:.1f} km")

            st.divider()
            col_c, col_t = st.columns([1, 1], gap="large")
            with col_c:
                st.subheader("Profil Atribut")
                st.caption("Perbandingan top-2 rekomendasi (skala
0-100)")
            draw_radar_chart(
                hasil.head(2),
                labels=["Rating", "Popularitas", "Harga", "Jarak
Hotel", "Jarak Pusat"],
                keys=["vote_average", "vote_count", "harga_tiket", "jarak_ke_hotel", "jarak_p
usat_km"],
                costs=[False, False, True, True, True],
            )
            with col_t:
                st.subheader("Peringkat Lengkap")
                st.caption(f"Top 10 dari {len(hasil)} destinasi yang
memenuhi filter")
            df_show =
hasil[["Ranking", "nama", "type", "vote_average", "harga_tiket",
"jarak_ke_hotel", "jarak_pusat_km", "Vector_V"]].head(10).copy()
            st.dataframe(
                df_show, use_container_width=True,
                hide_index=True,
                column_config={
                    "Ranking":
st.column_config.NumberColumn("#",
width="small"),
                    "nama":
st.column_config.TextColumn("Destinasi",
width="medium"),
                    "type":
st.column_config.TextColumn("Kategori",
width="small"),
                    "vote_average":
st.column_config.NumberColumn("Rating",
width="small",
format="%.1f"),

```

```

"harga_tiket":
st.column_config.NumberColumn("Harga      (Rp) ",          format="Rp      %d",
width="small"),

"jarak_ke_hotel":st.column_config.NumberColumn("Jarak          Hotel",
format="%.1f km", width="small"),

"jarak_pusat_km":st.column_config.NumberColumn("Jarak          Pusat",
format="%.1f km", width="small"),

"Vector_V":
st.column_config.NumberColumn("Skor      WP",              format="%.5f",
width="small"),

    },
)

with tab2:
    st.subheader("Peta Persebaran Wisata")
    st.caption("Merah = hotel Anda · Biru = rekomendasi destinasi
terdekat")
    if hotel_pilih and st.button("Tampilkan Peta", type="primary"):
        with st.spinner("Memuat peta.."):
            h_data = df_hotel[df_hotel["NAMA PENGINAPAN"] ==
hotel_pilih].iloc[0]
            m = folium.Map(location=[h_data["Latitude"],
h_data["Longitude"]], zoom_start=13)
            folium.Marker(
                [h_data["Latitude"], h_data["Longitude"]],
                popup=hotel_pilih, tooltip="Hotel Anda",
                icon=folium.Icon(color="red", icon="home",
prefix="fa"),
            ).add_to(m)
            peta_data = rekomendasi_wisata_dari_hotel(
                hotel_pilih, bobot_user, budget_maks, kat_pilih,
keyword_pilih, "descending", hari_pilih
            )
            for _, row in peta_data[peta_data["jarak_ke_hotel"] <=
radius_input].head(10).iterrows():
                folium.Marker(
                    [row["latitude"], row["longitude"]],
                    popup=f"#{int(row['Ranking'])}: {row['nama']}",
                    tooltip=row["nama"],
                    icon=folium.Icon(color="blue", icon="star",
prefix="fa"),
                ).add_to(m)
            st_folium(m, width=None, height=500, returned_objects=[])
        elif not hotel_pilih:
            st.info("Pilih hotel di panel kiri terlebih dahulu.")
        else:
            st.info("Tekan tombol di atas untuk memuat peta interaktif.")

with tab3:
    st.subheader("Analisis Sensitivitas Bobot")
    st.caption("Lihat pergeseran ranking jika bobot satu kriteria
dinaikkan +2.")
    if hotel_pilih:
        label_kri = {
            "vote_average":    "Rating",
            "vote_count":      "Popularitas",
            "harga_tiket":     "Harga Tiket",

```

```

        "jarak_ke_hotel": "Jarak dari Hotel",
        "jarak_pusat_km": "Jarak Pusat Kota",
    }
    kri_tes = st.selectbox(
        "Kriteria yang diuji",
        options=list(label_kri.keys()),
        format_func=lambda k: label_kri[k],
    )
    if st.button("Jalankan Analisis", type="primary"):
        with st.spinner("Menganalisis..."):
            tabel_sens =
hitung_analisis_sensitivitas(hotel_pilih, bobot_user, kri_tes, delta=2,
hari=hari_pilih)
            st.info(f"Bobot **{label_kri[kri_tes]}** dinaikkan +2.
Positif = ranking naik, negatif = turun.")
            st.dataframe(
                tabel_sens, use_container_width=True,
hide_index=True,
                column_config={
                    "nama":
st.column_config.TextColumn("Destinasi"),
                    "Rank_Awal":
st.column_config.NumberColumn("Rank Sebelum", width="small"),
                    "Rank_Baru":
st.column_config.NumberColumn("Rank Sesudah", width="small"),
                    "Perubahan_Posisi":
st.column_config.NumberColumn("Perubahan", width="small"),
                },
            )
        else:
            st.info("Pilih hotel di panel kiri terlebih dahulu.")

# MODE B
elif MODE_B:
    with tab1:
        if not wisata_pilih:
            st.info("Pilih destinasi tujuan di panel kiri.")
        else:
            info_w = df_wisata_final[df_wisata_final["nama"] ==
wisata_pilih].iloc[0]
            with st.container(border=True):
                st.caption("DESTINASI TUJUAN")
                st.subheader(info_w["nama"])
                ia, ib, ic, id_ = st.columns(4)
                ia.metric("Kategori", info_w.get("type", "-"))
                ib.metric("Rating",
f"{info_w['vote_average']:.1f} / 5")
                ic.metric("Hotel Terdekat",
f"{int(info_w.get('jumlah_hotel_terdekat', 0))}")
                id_.metric("Jarak Pusat",
f"{info_w['jarak_pusat_km']:.1f} km")

            st.subheader("Rekomendasi Hotel Terdekat")
            st.caption("Weighted Product · 4 kriteria: kelas, kapasitas,
jarak, estimasi waktu tempuh")

```

```

        if st.button("Cari Hotel Terbaik", type="primary",
use_container_width=True):
            with st.spinner("Menghitung rekomendasi hotel..."):
                hasil_hotel =
rekomendasi_hotel_dari_wisata(wisata_pilih, bobot_hotel, "descending")
                if filter_bintang_b:
                    hasil_hotel =
hasil_hotel[hasil_hotel["GOLONGAN_SCORE"].isin(filter_bintang_b)]

            if hasil_hotel.empty:
                st.warning("Tidak ada hotel yang sesuai filter kelas
yang dipilih.")
            else:
                juara_h = hasil_hotel.iloc[0]
                with st.container(border=True):
                    kol_badge, _ = st.columns([1, 4])
                    kol_badge.success("Hotel #1")
                    st.subheader(juara_h["NAMA PENGINAPAN"])
                    st.caption(f"{juara_h['GOLONGAN']} · Skor WP:
{juara_h['Vector_V']:.6f}")
                    h1, h2, h3, h4 = st.columns(4)
                    h1.metric("Golongan", juara_h["GOLONGAN"])
                    h2.metric("Jumlah Kamar",
f"{int(juara_h['JUMLAH KAMAR'])}")
                    h3.metric("Jarak ke Wisata",
f"{juara_h['jarak_ke_wisata']:.1f} km")
                    h4.metric("Estimasi Waktu",
f"{int(juara_h['estimasi_waktu_menit'])} menit")

                st.divider()
                col_c, col_t = st.columns([1, 1], gap="large")
                with col_c:
                    st.subheader("Profil Hotel")
                    st.caption("Perbandingan top-2 (skala 0-100)")
                    draw_radar_chart(
                        hasil_hotel.head(2),
                        labels=["Kapasitas", "Kelas", "Jarak", "Waktu
Tempuh"],
                        keys=["JUMLAH
KAMAR", "GOLONGAN_SCORE", "jarak_ke_wisata", "estimasi_waktu_menit"],
                        costs=[False, False, True, True],
                    )
                with col_t:
                    st.subheader("Peringkat Hotel")
                    st.caption(f"Top 10 dari {len(hasil_hotel)}
hotel")

                    df_h = hasil_hotel[[
                        "Ranking", "NAMA
PENGINAPAN", "GOLONGAN", "JUMLAH KAMAR",
"jarak_ke_wisata", "estimasi_waktu_menit", "Vector_V"
                    ]].head(10).copy()
                    st.dataframe(
                        df_h, use_container_width=True,
                        hide_index=True,
                        column_config={
                            "Ranking":
st.column config.NumberColumn("#",
width="small"),

```

```

        "NAMA PENGINAPAN":
st.column_config.TextColumn("Nama Hotel", width="medium"),
        "GOLONGAN":
st.column_config.TextColumn("Golongan", width="small"),
        "JUMLAH KAMAR":
st.column_config.NumberColumn("Kamar", format="%d", width="small"),
        "jarak_ke_wisata":
st.column_config.NumberColumn("Jarak", format="%.1f km",
width="small"),
        "estimasi_waktu_menit":
st.column_config.NumberColumn("Estimasi", format="%d mnt",
width="small"),
        "Vector_V":
st.column_config.NumberColumn("Skor WP", format="%.5f",
width="small"),
    },
)

with tab2:
    st.subheader("Peta Hotel Sekitar Wisata")
    st.caption("Merah = destinasi tujuan · Biru = hotel rekomendasi")
    if wisata_pilih and st.button("Tampilkan Peta", type="primary"):
        with st.spinner("Memuat peta..."):
            w_data = df_wisata_final[df_wisata_final["nama"] ==
wisata_pilih].iloc[0]
            m = folium.Map(location=[w_data["latitude"],
w_data["longitude"]], zoom_start=13)
            folium.Marker(
                [w_data["latitude"], w_data["longitude"]],
                popup=wisata_pilih, tooltip="Destinasi Tujuan",
                icon=folium.Icon(color="red", icon="flag",
prefix="fa"),
            ).add_to(m)
            for _, row in rekomendasi_hotel_dari_wisata(wisata_pilih,
bobot_hotel).head(10).iterrows():
                folium.Marker(
                    [row["Latitude"], row["Longitude"]],
                    popup=f"#{int(row['Ranking'])}: {row['NAMA
PENGINAPAN']}",
                    tooltip=row["NAMA PENGINAPAN"],
                    icon=folium.Icon(color="blue", icon="bed",
prefix="fa"),
                ).add_to(m)
            st_folium(m, width=None, height=500, returned_objects=[])
        elif not wisata_pilih:
            st.info("Pilih destinasi di panel kiri terlebih dahulu.")
        else:
            st.info("Tekan tombol di atas untuk memuat peta interaktif.")

# MODE C
elif MODE_C:
    with tab1:
        st.subheader("Paket Liburan Jogja")
        st.caption("Sistem memilihkan destinasi terbaik sesuai gaya
liburan, lengkap dengan opsi hotel terdekat.")

        if st.button("Racik Paket Liburan", type="primary",
use_container_width=True):

```

```

        with st.spinner("Menganalisis data wisata se-Jogja..."):
            st.session_state.hasil_paket = rekomendasi_wisata_global(
                bobot_wisata_global, kat_pilih, keyword_pilih,
                budget_min, budget_maks
            ).head(5)

        if not st.session_state.hasil_paket.empty:
            st.divider()
            st.caption("TOP 5 PAKET REKOMENDASI")

            for i, (_, w_row) in
enumerate(st.session_state.hasil_paket.iterrows()):
                with st.container(border=True):
                    rank_col, detail_col, hotel_col, aksi_col =
st.columns(
                    [0.5, 2, 3, 0.8], vertical_alignment="center"
                )
                    rank_col.subheader(f"#{i+1}")

                    with detail_col:
                        st.markdown(f"**{w_row['nama']}**")
                        st.caption(
                            f"{w_row.get('type', '-')} . "
                            f"Rating {w_row['vote_average']} . "
                            f"Rp {int(w_row['harga_tiket']):,} . "
                            f"{w_row['jarak_pusat_km']:.1f} km dari pusat
kota"
                        )

                    with hotel_col:
                        hasil_h =
rekomendasi_hotel_dari_wisata(w_row["nama"], bobot_hotel_global)
                        if filter_bintang_c:
                            hasil_h =
hasil_h[hasil_h["GOLONGAN_SCORE"].isin(filter_bintang_c)]
                            top3 = hasil_h.head(3)
                            if top3.empty:
                                st.caption("Tidak ada hotel sesuai kelas yang
dipilih.")
                            else:
                                for _, h in top3.iterrows():
                                    st.caption(
                                        f"{h['NAMA PENGINAPAN']} . "
                                        f"{h['jarak_ke_wisata']:.1f} km . "
                                        f"Kelas {int(h['GOLONGAN_SCORE'])}"
                                    )

                                if aksi_col.button("Detail", key=f"detail_{i}",
use_container_width=True):
                                    popup_detail_wisata(w_row)
                                else:
                                    st.info("Tekan tombol Racik Paket Liburan untuk memulai.")

            with tab2:
                st.subheader("Peta Destinasi Paket")
                st.caption("Hijau = destinasi dari paket yang sudah dihasilkan")
                if st.button("Tampilkan Peta", type="primary"):
                    with st.spinner("Memuat peta..."):

```



```

        m = folium.Map(location=[-7.7956, 110.3695],
zoom_start=11)
        if not st.session_state.hasil_paket.empty:
            for _, w in st.session_state.hasil_paket.iterrows():
                folium.Marker(
                    [w["latitude"], w["longitude"]],
                    popup=w["nama"], tooltip=w["nama"],
                    icon=folium.Icon(color="green", icon="star",
prefix="fa"),
                    ).add_to(m)
            st_folium(m, width=None, height=500, returned_objects=[])
        else:
            st.info("Hasilkan paket terlebih dahulu, lalu tekan tombol
untuk melihat peta.")

elif MODE_D:
    # --- TAB 1: PROFIL KELOMPOK ---
    with tab_profil:
        with st.container(border=True):
            st.subheader("Tim Pengembang SPK")
            st.write("Aplikasi Sistem Pendukung Keputusan (Metode
Weighted Product) ini dikembangkan oleh:")
            st.markdown("- **Raihan Buono Putra**")
            st.markdown("- **Reinnent Rasika Z**")
            st.caption("Praktikum Sistem Pendukung Keputusan (SCPK) -
2026")

    # TAB 2: DATASET MENTAH
    with tab_data:
        st.subheader("1. Dataset Wisata Gabungan (Fitur SPK)")
        st.caption("Gabungan data dasar wisata dengan ekstraksi fitur
geospasial.")
        st.dataframe(df_wisata_final, use_container_width=True,
height=250)

        st.subheader("2. Dataset Pariwisata (Fitur Ekstraksi Jarak)")
        st.caption("Hasil perhitungan Haversine dari notebook
pre-processing.")
        st.dataframe(df_pariwisata, use_container_width=True, height=250)

        st.subheader("3. Dataset Hotel (Cleaned)")
        st.caption("Data hotel bersih yang siap digunakan untuk kalkulasi
jarak.")
        st.dataframe(df_hotel, use_container_width=True, height=250)

    # --- TAB 3 ---
    with tab_grafik:
        st.info("Visualisasi Exploratory Data Analysis (EDA) menggunakan
Seaborn dan Matplotlib.")
        c1, c2 = st.columns(2)

        # Grafik 1: Bar Chart (Distribusi Kategori Wisata)
        with c1:
            st.write("**1. Distribusi Kategori Wisata di Yogyakarta**")
            fig1, ax1 = plt.subplots(figsize=(6, 4))
            sns.countplot(data=df_wisata_final, y='type',
order=df_wisata_final['type'].value_counts().index, palette='viridis',
ax=ax1)
            ax1.set_xlabel("Jumlah Destinasi")

```

```

        ax1.set_ylabel("Kategori")
        st.pyplot(fig1)

# Grafik 2: Pie Chart (Komposisi Kelas Hotel)
with c2:
    st.write("**2. Komposisi Ketersediaan Kelas Hotel**")
    fig2, ax2 = plt.subplots(figsize=(6, 4))
    hotel_counts = df_hotel['GOLONGAN'].value_counts().head(5)
    ax2.pie(hotel_counts, labels=hotel_counts.index,
            autopct='%1.1f%%', colors=sns.color_palette('pastel'))
    st.pyplot(fig2)

    st.divider()

# Grafik 3: Scatter Plot (Korelasi Harga & Rating)
    st.write("**3. Pemetaan Harga Tiket (Weekday) vs Rating
Wisata**")
    fig3, ax3 = plt.subplots(figsize=(10, 4))
    sns.scatterplot(data=df_wisata_final, x='htm_weekday',
                    y='vote_average', hue='type', palette='Set2', alpha=0.8, ax=ax3)
    ax3.set_xlabel("Harga Tiket (Rp)")
    ax3.set_ylabel("Rating (★)")
    ax3.legend(title='Kategori', bbox_to_anchor=(1.05, 1), loc='upper
left')
    st.pyplot(fig3)

# wp.py
import numpy as np
import pandas as pd

def hitung_jarak_haversine(lat1, lon1, lat2, lon2):
    """Menghitung jarak koordinat geografis (KM)"""
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) *
np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    return c * 6371

def core_weighted_product(df_alternatif, bobot, jenis_kriteria,
sort_order='descending'):
    """Engine utama rumus WP dengan fitur shortcut Ascend/Descend"""
    if df_alternatif.empty:
        return df_alternatif

    df_wp = df_alternatif.copy()

    # 1. Normalisasi Bobot
    total_bobot = sum(bobot.values())
    bobot_normal = {k: v / total_bobot for k, v in bobot.items()}

    # 2. Hitung Vektor S
    S = np.ones(len(df_wp))
    for kriteria in bobot.keys():
        w = bobot_normal[kriteria]
        if jenis_kriteria[kriteria] == 'cost':
            w = -w

```

```

nilai_kolom = df_wp[kriteria].replace(0, 0.01)
S *= nilai_kolom ** w

df_wp['Vector_S'] = S

# 3. Hitung Vektor V & Ranking
df_wp['Vector_V'] = df_wp['Vector_S'] / df_wp['Vector_S'].sum()
df_wp['Ranking'] = df_wp['Vector_V'].rank(ascending=False,
method='min')

# Shortcut Ascending / Descending
is_ascending = True if sort_order == 'ascending' else False
return df_wp.sort values(by='Vector_V', ascending=is_ascending)

```

Jogja Tourism — Recommender

Hotel: 551 | Wisata: 479

Paket Liburan Jogja

Sistem memilihkan destinasi terbaik sesuai gaya liburan, lengkap dengan opsi hotel terdekat.

TOP 5 PAKET REKOMENDASI

No	Destinasi	Rating	Jarak dari pusat kota	Hotel Rekomendasi	Jarak	Kelas
#1	Tugu	Budaya dan Sejarah - Rating 4.8 - Rp 0 - 0.0 km dari pusat kota	Bhinneka Hotel	0.1 km	Kelas 1	Detail
			Kumbokarmo Utama Hotel	0.1 km	Kelas 1	
			Pancowinatan Hotel	0.1 km	Kelas 1	
#2	Masjid Gedhe Kauman	Budaya dan Sejarah - Rating 4.8 - Rp 0 - 2.3 km dari pusat kota	Mitra Hotel	0.2 km	Kelas 1	Detail
			Arayan Residence Spariah	0.3 km	Kelas 2	
			Cabin Hotel	0.3 km	Kelas 1	
#3	Museum Sandi	Budaya dan Sejarah - Rating 4.7 - Rp 0 - 0.3 km dari pusat kota	Pop Sangaji Hotel	0.5 km	Kelas 2	Detail
			Graha Kinasih Kotabaru	0.3 km	Kelas 1	
			Citra Dream Hotel	0.6 km	Kelas 2	
Bundaran UGM			Indraloka Heritage Homestay	0.4 km	Kelas 1	Detail

Detail Destinasi

Tugu

Kategori: Budaya dan... | Rating: 4.8 / 5

Harga Tiket: Rp 0 | Jarak Pusat Kota: 0.0 km

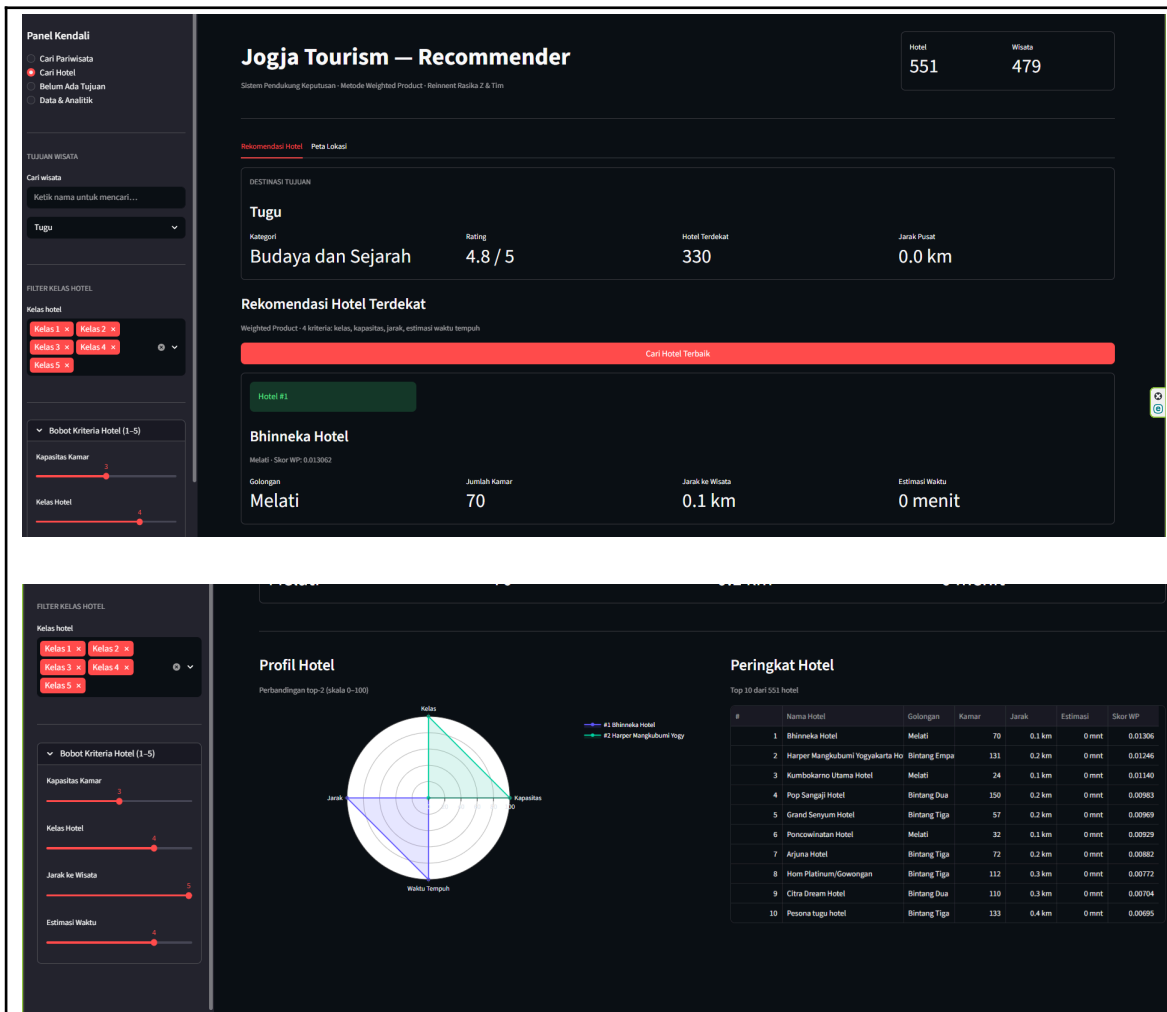
Ulasan: 13,330 orang | Hotel Terdekat: 330 hotel

[Ingin melihat rekomendasi hotel khusus untuk destinasi ini?](#)

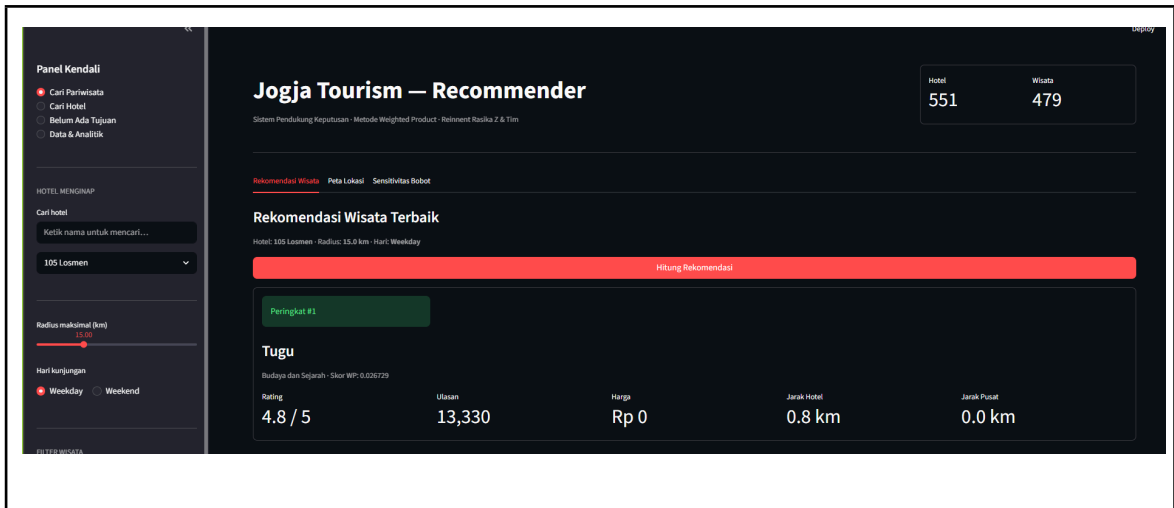
[Cari Hotel untuk Destinasi Ini](#)

Kumbokarmo Utama Hotel - 0.1 km - Kelas 1

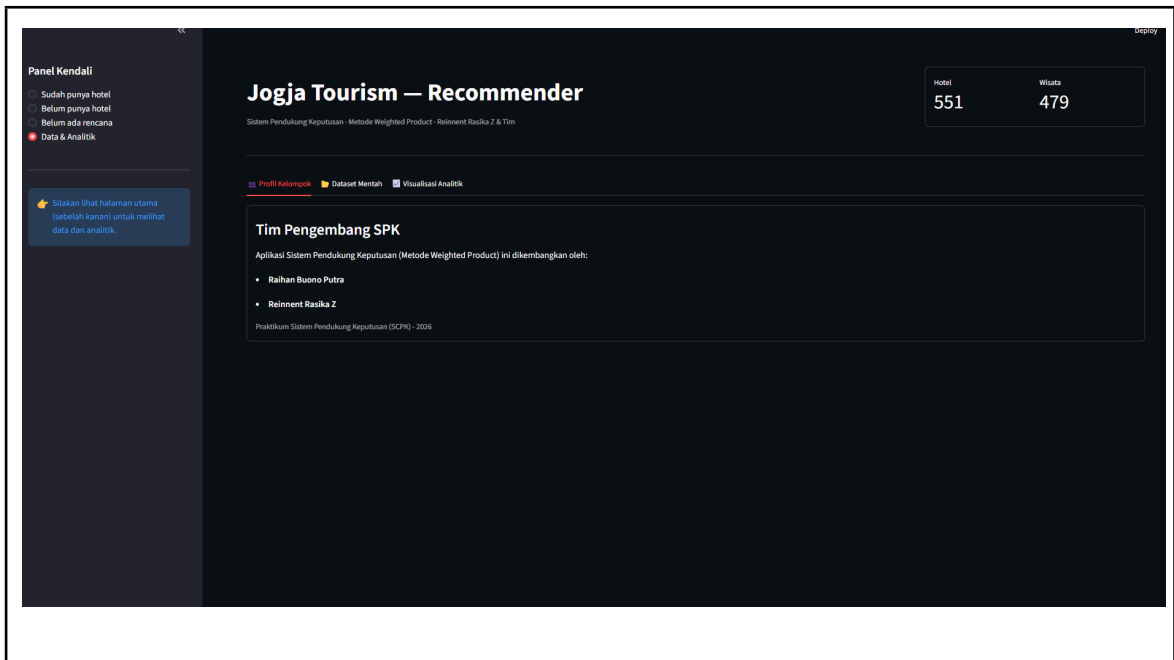
Gambar Mode C dimana belum punya tujuan

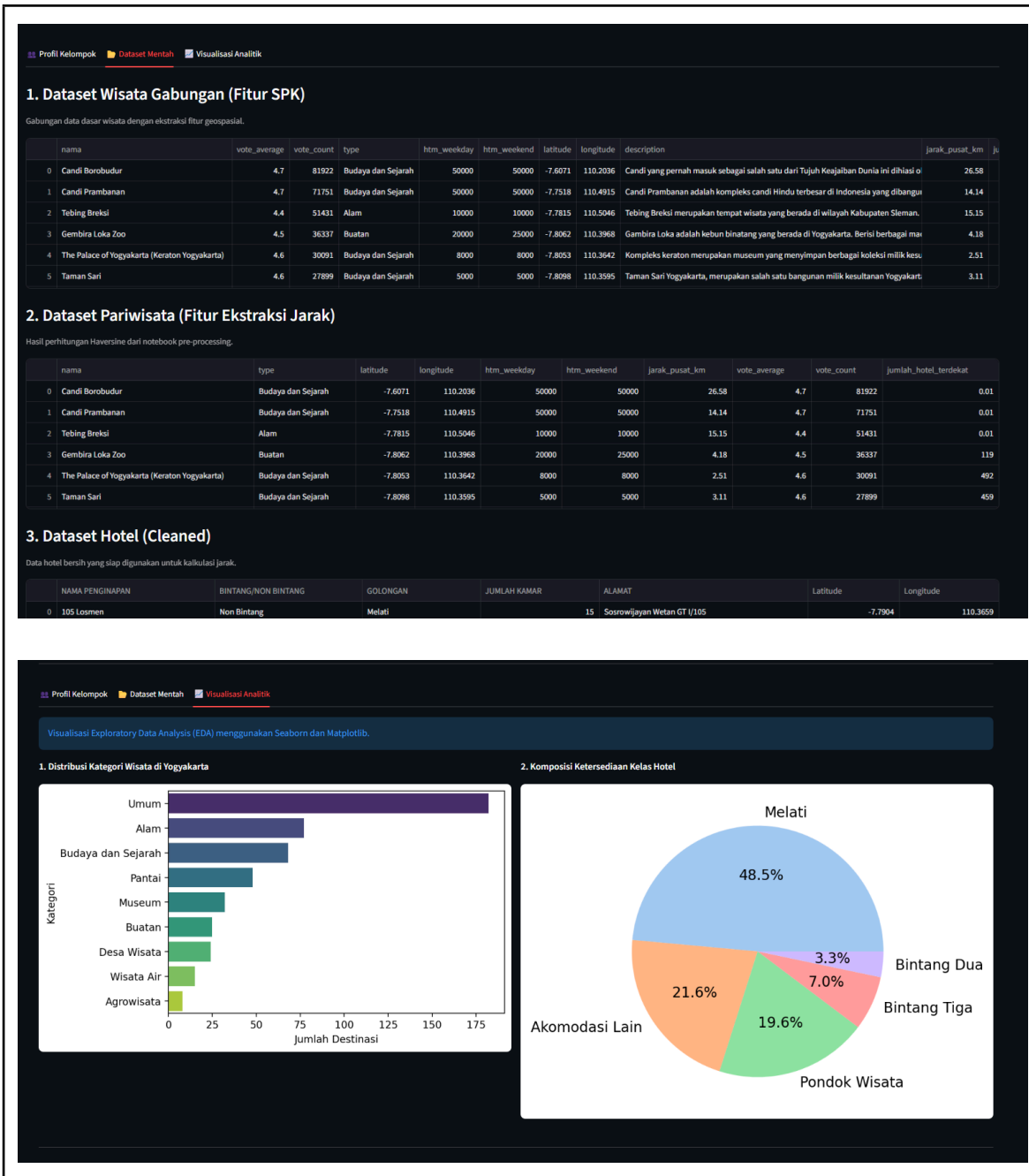


Gambar Mode Bdimana belum punya hotel



Gambar Mode A dimana belum punya pariwisata





Gambar mode D Analisa dan data

BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026
		Minggu

		1	2	3	4	dst
1	Data Proceccing			V		
2	Backend & Scpk Algorsm				V	
3	Front end				V	
4	Project Manager & Documentation				V	
dst						

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Raihan Buono	123240090	- Tugas 1 - Tugas 3
2	Reinnent Rasika Z	123240242	- Tugas 2 - Tugas 4

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem pada proyek "Jogja Tourist Recommendation", dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Implementasi Metode WP yang Berhasil:** Sistem Pendukung Keputusan (SPK) berhasil dibangun menggunakan metode *Weighted Product* (WP). Metode ini terbukti mampu menyeleksi dan merangking destinasi wisata serta hotel di Yogyakarta secara objektif dan matematis berdasarkan kriteria-kriteria yang saling bertolak belakang (seperti memaksimalkan *rating* sekaligus meminimalkan harga dan jarak).
2. **Analisis Spasial yang Akurat:** Integrasi algoritma *Haversine* di dalam sistem berfungsi dengan baik untuk menghitung jarak geospasial secara langsung antara titik koordinat lokasi menginap pengguna dengan titik lokasi destinasi wisata.
3. **Fleksibilitas dan Interaktivitas Tinggi:** Sistem yang dikembangkan berbasis *web* (menggunakan kerangka kerja Streamlit) mampu memberikan interaktivitas tinggi. Pengguna dapat mengubah bobot preferensi (seperti mengutamakan anggaran hemat atau jarak dekat) secara *real-time* dan melihat perubahan hasil rekomendasi seketika.
4. **Dukungan Visualisasi Keputusan:** Keberadaan fitur visualisasi tambahan seperti pemetaan spasial (peta interaktif) dan *Radar Chart* sangat efektif dalam membantu pengambil keputusan (wisatawan) untuk membandingkan karakteristik dan sebaran lokasi dari masing-masing alternatif rekomendasi.

4.3 Saran

Untuk perluasan dan pengembangan sistem di masa mendatang agar menjadi lebih komprehensif, beberapa hal yang dapat disarankan adalah:

1. **Penerapan *Machine Learning*:** Mengingat arsitektur saat ini sudah terstruktur dengan baik, sistem dapat dikembangkan lebih lanjut menjadi *hybrid recommender system*. Metode WP dapat digabungkan dengan algoritma *Machine Learning* seperti *Collaborative Filtering* untuk memberikan rekomendasi berdasarkan riwayat preferensi pengguna lain, atau menggunakan *Natural Language Processing* (NLP) untuk menganalisis sentimen *review* teks dari pengunjung sebelumnya.
2. **Integrasi Data Lalu Lintas *Real-Time*:** Perhitungan jarak saat ini menggunakan pendekatan garis lurus (*Haversine*). Ke depannya, sistem dapat diintegrasikan dengan API eksternal (seperti Google Maps API atau Mapbox) untuk menghitung jarak tempuh aktual, rute jalan, dan estimasi waktu berkendara berdasarkan kondisi lalu lintas terkini.
3. **Perluasan Ekosistem Kriteria:** Dataset dan kriteria dapat diperluas dengan memasukkan elemen pariwisata lainnya yang sering dicari wisatawan di Yogyakarta, seperti rekomendasi titik kuliner lokal, pusat oleh-oleh, ketersediaan

lahan parkir, serta aksesibilitas transportasi umum (seperti kedekatan dengan halte Trans Jogja).

4. **Pengembangan ke Platform *Mobile*:** Wisatawan umumnya merencanakan dan mengevaluasi perjalanan mereka melalui ponsel cerdas saat sedang bepergian. Mengembangkan sistem ini menjadi aplikasi *mobile native* (Android/iOS) akan sangat meningkatkan aksesibilitas dan kenyamanan pengguna.
5. **Fitur *Outbound Booking*:** Sistem dapat dikembangkan agar tidak hanya berhenti pada tahap pemberian rekomendasi, tetapi juga menyediakan fitur *redirect* atau integrasi pemesanan langsung ke platform *Online Travel Agent* (OTA) untuk pemesanan kamar hotel dan pembelian tiket destinasi.

DAFTAR PUSTAKA

Pemerintah Kota Yogyakarta. (2026). *Dataset Hotel*. Portal Data Terpadu Pemerintah Kota Yogyakarta. Diakses pada [Isi Tanggal Lu Akses, misal: 26 Mei 2026], dari <https://dataset.jogjakota.go.id/dataset/?tags=hotel>

Rizqiawan, F. (n.d.). *Dataset Wisata Jogja Sekitar* [Dataset]. Kaggle. Diakses pada [Isi Tanggal Lu Akses, misal: 26 Mei 2026], dari <https://www.kaggle.com/datasets/farisrizqiawan/dataset-wisata-jogja-sekitar>